Uma loja pequena.
Três formas de processar.

C# do pedido à persistência. Uma comparação experimental de execução sequencial, paralela e distribuída.

5.000

pedidos por lote
semente fixa 42

7 × 3

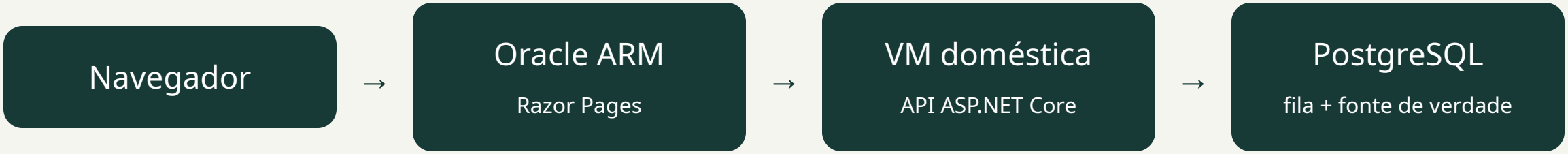
cenários × repetições
105.000 pedidos na campanha

1 regra

estoque consistente
pagamento fictício

Fonte: Prof. Eng. Nicolas Vogiantzis, slides 16–17, e roteiro do grupo.

Os nós e as mensagens.



HTTP privado

Site → API via Tailscale, com chave. 202 retorna o ID; GET consulta o estado.

Fila persistida

Workers disputam pedidos pelo protocolo PostgreSQL. Sem broker externo.

Independência

O site carrega sem a API. Não há portas abertas no roteador doméstico.

Topologia de implantação preparada. Nesta entrega os testes foram locais em Docker amd64; Oracle/Tailscale reais ainda precisam de validação.

Primeiro, a referência sequencial.

Uma thread de pedidos

Um processo eleito assume a fila. Cada pedido termina antes de o próximo começar.

Validar → reservar → calcular → simular pagamento → concluir.

Resultados são consolidados somente após todos os pedidos chegarem ao estado final.

```
var digest = Workload.Compute(
job.Seed, job.Iterations);

store.Complete(job, digest,
watch.Elapsed.TotalMilliseconds,
thread);
```

A mesma rotina C# é chamada nos três modos. SHA-256 em cadeia é CPU real; atraso visual fica desligado no benchmark.

Paralelo: dividir pedidos, preservar regras.

2

threads dedicadas
no mesmo processo

4

threads dedicadas
no mesmo processo

8

threads dedicadas
no mesmo processo

Cada thread retira o próximo pedido disponível. CPU pode executar simultaneamente; estoque continua protegido por transação.

Thread não é núcleo e Task não é thread exclusiva. A VM planejada tem 4 vCPU: oito threads podem aumentar contenção e saturação.

Evidências: thread ID por pedido, log de início/fim e pico de pedidos ativos por processo. O ambiente de desenvolvimento expõe 3 processadores lógicos.

Distribuído: processos independentes.

worker-1	worker-2	worker-3
1 pedido por vez	1 pedido por vez	1 pedido por vez

Um, dois ou três contêineres compartilham a fila persistida, sem compartilhar memória de aplicação.

Um slot por processo. Pedido com posse vencida pode ser recuperado por outro worker.

Três contêineres na mesma máquina dividem CPU, memória e banco: não equivalem a três servidores físicos.

Opcional preparado: worker ARM na Oracle + workers x64 na VM. Não misturar essa topologia com a comparação básica em um único host.

Consistência antes da velocidade.

Estoque

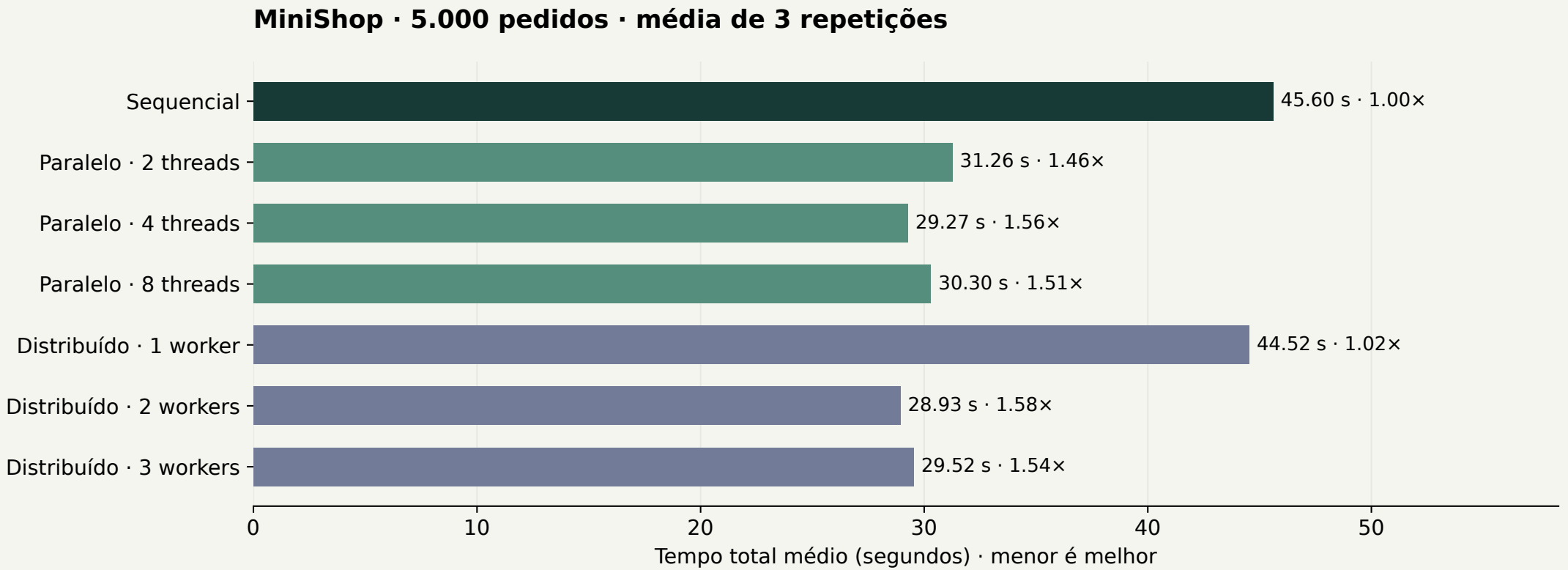
- Reserva e redução em uma transação.
- Produtos bloqueados na mesma ordem.
- Constraint impede estoque negativo.
- Pagamento recusado devolve a reserva.

Duplicatas e falhas

- Idempotency-Key + hash do carrinho.
- Lease de 15 s, renovação a cada 2 s.
- Novo token invalida a tentativa antiga.
- Evento final único por pedido.

Computação pode ser repetida durante recuperação; efeitos persistidos são únicos. PostgreSQL central é um ponto único de falha, protegido por persistência e backup, sem failover automático.

O resultado medido.



Histórico: Linux amd64 / Docker / 3 processadores lógicos disponíveis. 5.000 iterações SHA-256 por pedido; atraso = 0. O padrão do projeto é 50.000. Não são tempos da Oracle ou do Ryzen.

Derrubar um nó e conferir os efeitos.

24

pedidos na demonstração
3 workers inicialmente

2

tentativas no pedido
recuperado após a queda

1

evento final por pedido
nenhuma venda duplicada

Worker-2 foi interrompido abruptamente com reserva ativa. Os sobreviventes continuaram e recuperaram o trabalho após o prazo de posse.

Evidência histórica real: 2026-10-20T16:00:57Z · pedido 5fc1ec51-712a-48d8-9acf-39f0ecbf345f.

Também verificados: 20 compras sobre 5 unidades, sobrecarga com 503, API indisponível com site carregando e reinício do banco/API preservando pedidos pendentes.

Comparar, conferir e explicar.

Cenário	Média (s)	Speedup
Sequencial	45.602	1.00×
Paralelo · 2 threads	31.256	1.46×
Paralelo · 4 threads	29.265	1.56×
Paralelo · 8 threads	30.296	1.51×
Distribuído · 1 worker	44.525	1.02×
Distribuído · 2 workers	28.928	1.58×
Distribuído · 3 workers	29.523	1.54×

Conclusões do experimento

O ganho vem de tarefas simultâneas, mas é limitado por CPU, SQL, comunicação e coordenação.

Três repetições e o mesmo lote. Todos os estados finais, pagamentos e estoques precisam coincidir.

Repetir na VM final antes da aula. Se a infraestrutura cair, apresentar estes dados como históricos, sem simular execução ao vivo.